



Programación Orientada a Objetos

Paso de mensajes entre Objetos

Estudiante:

José Alessandro Torres Parodi.

Docente:

Ing. José Durán García.

Managua, Nicaragua

1. Sistema de Pedidos para tienda

- ¿Qué objeto inicia la comunicación?

El objeto que comienza la comunicación es el objeto de pedido, en este caso desde la clase de Cliente con el método de realizarPedido el cual como parámetros recibe un objeto de tipo tienda, un string producto para recibir el nombre del producto y int cantidad para la cantidad de productos que se realizará en el pedido dentro de este método se toma el método que pertenece a la clase Tienda de procesarPedido que inicializa un objeto de tipo Pedido y este es el responsable de recibir todos los datos para luego mostrarlos en pantalla.

- ¿Qué método representa el paso de mensaje?

el paso de mensaje ocurre en la línea: cliente.realizarPedido(tienda, pedido.getProducto(), pedido.getCantidad());

Y de manera más profunda (dentro de la clase Cliente), el paso de mensaje clave es: tienda.procesarPedido(this, producto, cantidad);

Explicación: En POO, un "mensaje" es simplemente la llamada a un método de un objeto. Aquí, el objeto Cliente le envía un "mensaje" al objeto Tienda para que ejecute una acción.

- ¿Qué parámetros se envían?

this (El objeto actual): Se envía la instancia completa del Cliente. Esto permite que la tienda sepa quién es el comprador sin tener que pasar nombre, ID y dirección por separado.

producto (String): El nombre o identificador del artículo que se desea comprar.

cantidad (int): El número de unidades solicitadas.

- Rol de cada clase

Clase	Rol Principal	Responsabilidad
Cliente	Actor / Emisor	Representa al usuario del sistema. Su rol es iniciar la acción de compra (realizarPedido) y proveer su propia información.
Pedido	Entorno de Datos (DTO)	Es una clase "contenedor". Su rol es almacenar temporalmente la información de qué se está pidiendo y cuánto, para transportarla entre el cliente y la tienda.
Tienda	Controlador / Receptor	Es la que contiene la "lógica de negocio". Su rol es procesar la solicitud, validar stock, calcular totales y confirmar si el pedido se puede realizar o no.
Main	Orquestador	No es una clase de negocio, su único rol es gestionar la interfaz de usuario (el menú) y conectar las piezas del sistema.

2. Sistema de cálculo de salarios

Diferencia entre métodos con y sin retorno

La diferencia se ve claramente en cómo usas el resultado de cada acción.

- **Métodos sin retorno (void):** Su función es **ejecutar una acción final**, como imprimir en pantalla. No devuelven un valor que puedas guardar o usar en una operación matemática.
 - *Ejemplo en tu código:* `mostrarDatos()`. Este método solo hace el `System.out.println`. Si intentas hacer `double x = mostrarDatos();`, el código fallará porque no hay nada que "regresar".
- **Métodos con retorno (double, boolean):** Su función es **calcular o procesar un dato** y "entregarlo" a quien lo llamó.
 - *Ejemplo en tu código:* `calcularBono()`. Este método devuelve un número decimal. Gracias a que tiene retorno, puedes usarlo dentro de otro método: `salarioBase + calcularBono()`.

Reutilización de métodos

La reutilización consiste en **no escribir la misma lógica dos veces**. En tu programa de salarios, esto ocurre en la clase `Empleado`:

Java

```
public double calcularSalarioTotal() {  
  
    return salarioBase + calcularBono(); // Aquí estamos reutilizando  
  
}
```

¿Por qué es importante? Si el día de mañana la empresa decide que el bono ya no es del **10%** sino del **15%**, solo tienes que cambiar una línea en `calcularBono()`.

El beneficio: El método `calcularSalarioTotal()` se actualizará automáticamente porque "confía" en el resultado de `calcularBono()`. Esto evita errores humanos y hace que el código sea más fácil de mantener.

Separación de responsabilidades

Este es un principio de diseño donde cada clase y método se encarga de **una sola cosa**. En tu programa, las responsabilidades están divididas así:

A. Responsabilidad a nivel de Clases:

- **Empleado:** Solo sabe sobre sus propios datos (nombre, sueldo) y cómo calcular lo que le pertenece a él. No sabe nada de otros empleados.
- **Empresa:** No sabe cómo se calcula el bono de un empleado, pero sabe **administrar la lista** de todos ellos y sumarlos para obtener la planilla total.
- **Main:** No hace cálculos; su única responsabilidad es **hablar con el usuario** (mostrar el menú y recibir datos).

B. Responsabilidad a nivel de Métodos:

`aplicaDescuento()`: Sólo decide si un salario es alto o bajo (devuelve `true` o `false`). No imprime el mensaje.

`verificarDescuento(Empleado e)` (en el Main): Es el encargado de decidir **qué mensaje mostrar** al usuario basándose en lo que el método anterior le respondió.